

Labwork 4: Threads

Philippe Olivier Schriqui-Duque
ID: 2640042

Introduction

In this labwork, I improved the CUDA grayscaling program from Labwork 3 by switching from a 1D grid to a 2D grid of thread blocks. Instead of reshaping the image into a long 1D array, the image keeps its natural 2D shape (3840×2880). I measured the speedup compared to the CPU, tested several 2D block configurations over 100 runs to compute the standard deviation, and answered the exercises from the slides.

Implementation

CPU

On the CPU side, I iterate through the image using two nested loops over height (y) and width (x), computing the average of the RGB channels for each pixel:

```
for y in range(h):
    for x in range(w):
        gray = (int(img[y, x, 0]) +
                int(img[y, x, 1]) +
                int(img[y, x, 2])) // 3
        gray_cpu[y, x, 0] = gray
        gray_cpu[y, x, 1] = gray
        gray_cpu[y, x, 2] = gray
```

GPU (2D Kernel)

For the GPU, each thread calculates its 2D coordinates (x, y) with its thread index, block index, and block dimension. The `if` condition ensures we stay within the image width and height:

```
@cuda.jit
def grayscale(src, dst):
    x = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    y = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    if x < src.shape[1] and y < src.shape[0]:
        gray = (int(src[y, x, 0]) +
                int(src[y, x, 1]) +
                int(src[y, x, 2])) // 3
        dst[y, x, 0] = gray
        dst[y, x, 1] = gray
        dst[y, x, 2] = gray
```

The launch function calculates a 2D `gridSize` tuple to cover both dimensions:

```
def gray_gpu(blockSize):
    bx, by = blockSize
    gridSize = ((w + bx - 1) // bx, (h + by - 1) // by)
    devSrc = cuda.to_device(img)
    devDst = cuda.device_array((h, w, 3), np.uint8)
    grayscale[gridSize, blockSize](devSrc, devDst)
    return devDst.copy_to_host()
```

Results

For a single run with a block size of (8,8):

```
CPU time: 25.52 s
GPU Time: 0.0344 s
speedup: 741 x
```

Compared to Labwork 3 (CPU 18.42 s, GPU 0.0368 s, speedup 501x), the GPU compute time is very similar. The main advantage of the 2D grid is that the code is much cleaner and does not require flattening or reshaping the image on the host.

Block size vs speedup

I ran an experiment testing 7 different 2D block sizes. For each block size, I executed the kernel 100 times to compute the average speedup and the standard deviation:

```
block size 8x8   : speedup = 1129.95x +/- 172.13x
block size 16x8  : speedup = 1137.45x +/- 167.75x
block size 16x16 : speedup = 1104.04x +/- 169.59x
block size 32x8  : speedup = 1124.96x +/- 167.08x
block size 16x32 : speedup = 1143.53x +/- 171.43x
block size 32x16 : speedup = 1153.18x +/- 157.74x
block size 32x32 : speedup = 1145.88x +/- 167.95x
```

The graph is saved in `blocksize.vs.speedup.png`.

Looking at the results, all mean speedups are around 1100x to 1150x, with a large standard deviation of around 160x to 170x. Because the error ranges overlap significantly between all block sizes, there is no real correlation between block size and performance. This happens because memory transfer over the PCIe bus dominates the overall time, so changing the 2D block layout does not noticeably affect total execution time.

Exercises

Exercise 1

Given a GPU with max 512 threads/block, 1024 threads/SM, 8 blocks/SM, and 32 threads/warp:

- $32 \times 32 = 1024$ threads/block, which exceeds the max 512 limit (invalid).
- $8 \times 8 = 64$ threads/block. With max 8 blocks per SM, $8 \times 64 = 512$ threads/SM (50% occupancy).
- $16 \times 16 = 256$ threads/block. 4 blocks $\times 256 = 1024$ threads/SM, reaching 100% occupancy.

The best configuration is 16×16 .

Exercise 2

Given an SM with max 1536 threads and 4 blocks:

- 128 threads/block: $4 \times 128 = 512$ threads.
- 256 threads/block: $4 \times 256 = 1024$ threads.
- 512 threads/block: $3 \times 512 = 1536$ threads (100% capacity).
- 1024 threads/block: $1 \times 1024 = 1024$ threads.

The configuration with the most threads in the SM is 512 threads/block.

Exercise 3

For a vector length of 2000, 1 output per thread, and block size of 512:

- Number of blocks = $\lceil 2000/512 \rceil = 4$ blocks.
- Total threads in grid = $4 \times 512 = 2048$ threads.

Conclusion

Using 2D thread blocks makes CUDA image kernels easier to write and read. As shown by the experiment, block size does not significantly impact performance for simple grayscaling because host-device memory transfers remain the bottleneck.